AgenticCareerBoost

Sprint S-000: Agentic OS Bootstrap

Engineering Documentation

Dídac Llorens
github.com/DidacLL

April 2026 — v1.0

Contents

Chapter 1

Preface

Key takeaway: This document records the complete architecture, rationale, and implementation of a path-based, model-agnostic multiagent operating system built inside a single Git repository. It is the engineering proof that the system works, written for two audiences: a recruiter who needs fast evidence of capability, and an engineering peer who values visible craft.

1.1 Mission

The AgenticCareerBoost project exists to rebuild a public technical profile through visible, agentic engineering work. It converts scattered real capability into recruiter-readable, peer-inspectable proof. The repository itself is the primary artifact: every operational rule is a Markdown file with git history, every sprint produces documented closure artifacts, and the architecture is designed to be understood by any LLM that can read files and follow paths.

1.2 Audience

Recruiters
: Need fast proof of capability and role-fit. They will scan this document's diagrams, the file manifest, and the sprint closure checklist. The table of contents and chapter openers are written for them.

Engineering peers
: Value visible craft, systems thinking, and tooling depth. They will read the rationale sections, inspect the TikZ diagrams, and follow the truth-hierarchy logic. The "common pitfalls" boxes are written for them.

1.3 Reading path

Chapters 2–3 cover the big picture: how agents navigate the repository and why conflicting information is resolved deterministically. Chapters 4–9 walk through every subsystem in the order an agent would encounter them. Chapter 10 covers CI/CD automation. Chapter 11 restates the invariants. Appendix A lists every file.

Common pitfall: Do not read this document linearly if you only want to understand one subsystem. Use the table of contents and jump to the relevant chapter.

Chapter 2

Architectural Overview

Key takeaway: The system is path-based: a single entrypoint file (`AGENTS.md`) dispatches agents to the specific files they need. No mega-prompt, no vendor lock-in, no model-specific tricks.

2.1 Routing map

Figure 2.1 shows the complete information flow from a user prompt through the agent system to published outputs.

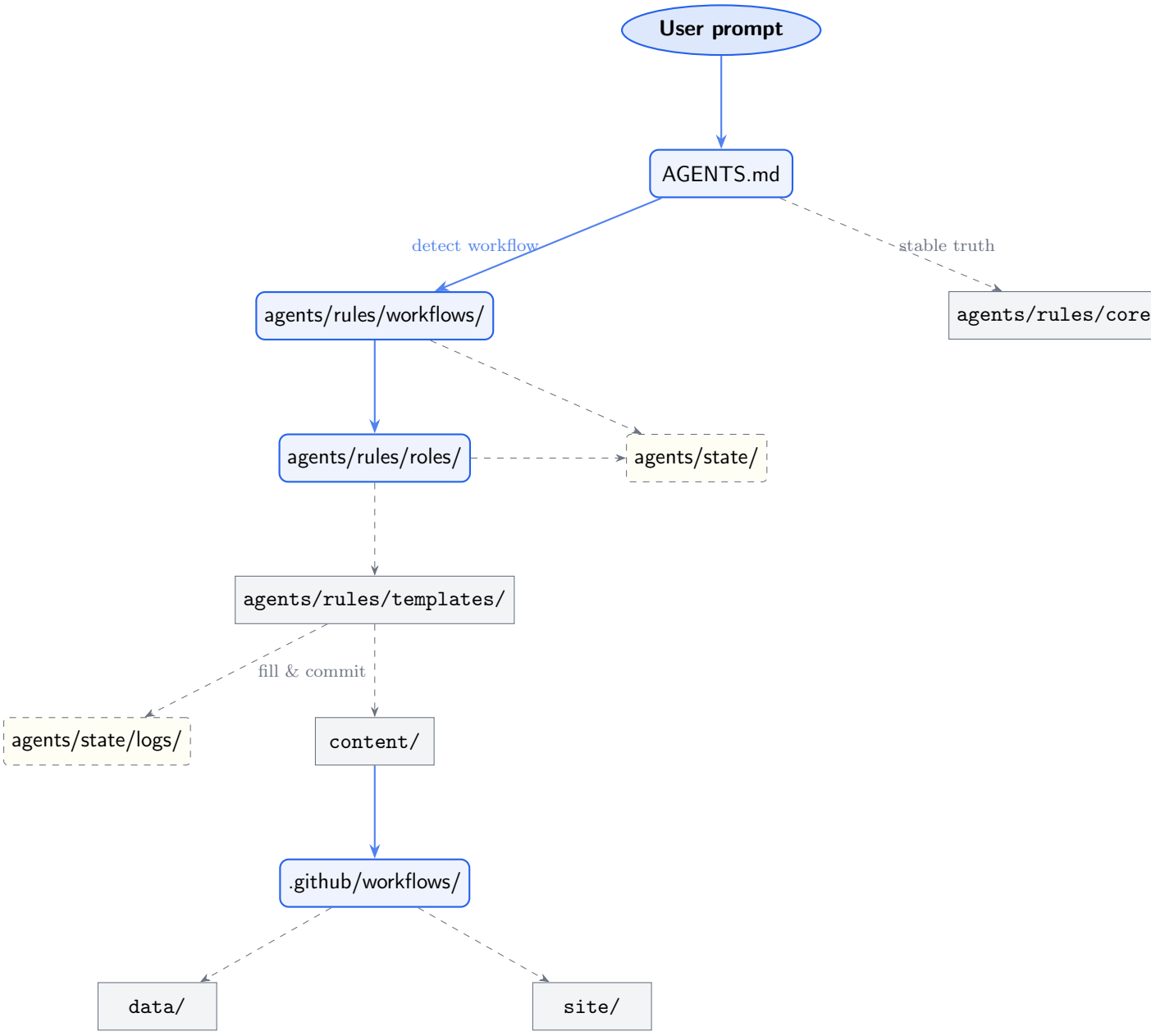


Figure 2.1: Repository routing map: from user prompt to published outputs.

2.2 Design principles

The architecture obeys six invariants:

1. **Token cost** — agents load only the 1–3 files needed per turn, not a monolithic prompt.
2. **Conflict containment** — volatile files can be wrong without corrupting canonical truth; a strict hierarchy resolves precedence.
3. **Model agnosticism** — any LLM that reads Markdown and follows file paths works. No vendor-specific prompt engineering in canonical files.
4. **Auditability** — every rule is a file with git history. Reviewers diff changes instead of re-reading prose.
5. **Parallel agents** — different agents open different files without racing on a shared prompt buffer.
6. **Graceful failure** — a missing file forces the agent to stop and ask, rather than hallucinate structure.

2.3 The entrypoint: AGENTS.md

`AGENTS.md` is the machine-readable entrypoint. It contains:

- A truth-priority list (one-liner summary of the full hierarchy).
- A workflow dispatch table mapping keywords to file paths.
- A role index mapping agent names to their contract files.
- State pointers to volatile files.
- A rationale footer explaining why path-based beats mega-prompt.

An agent arriving at the repository reads this single file, identifies the active workflow keyword, and navigates to exactly the files it needs. Total context loaded: typically under 3 KB.

> **Common pitfall:** Never add content to `AGENTS.md` that does not serve the routing function. It is a dispatch table, not documentation.

Chapter 3

Truth Hierarchy and Token Rationale

Key takeaway: When information conflicts, the system resolves it using a strict six-level priority. This eliminates ambiguity without requiring agents to load everything.

3.1 The six levels

Priority	Source	Mutability
1	Direct user prompt	Per-session
2	`agents/rules/core/`	Stable — change only via system-review
3	`agents/rules/workflows/`	Stable — change only via system-review
4	`agents/rules/roles/`	Stable — change only via system-review
5	`agents/state/`	Volatile — updated every sprint/hotfix
6	Backlog, logs, summaries	Volatile — may be outdated

3.2 Resolution rules

- Agent-generated logs are never canonical truth.
- If a volatile file contradicts a stable file, the stable file wins.
- If two stable files at the same level contradict, escalate to the user.
- If a referenced path does not exist, **stop and ask** — do not fabricate content or structure.

3.3 Token rationale

The hierarchy exists because loading all files into a single context window is wasteful and error-prone. An agent running the `chat` workflow needs only `agents/rules/core/` and `agents/state/current.md` — under 3 KB of context. An agent running `sprint` adds workflow and role contracts but never loads logs or summaries unless explicitly needed.

This design keeps per-turn cost low, makes conflicts deterministic, and allows the system to scale to dozens of files without degrading agent performance.

Common pitfall: Never flatten the hierarchy into a single file "for convenience." The separation *is* the conflict-resolution mechanism.

Chapter 4

Canonical Truth: agents/rules/core/

Key takeaway: Six files define the stable ground truth of the project. They are read-only during sprints and can only be modified through a system-review workflow. Every other file in the repository defers to them.

4.1 mission.md

agents/rules/core/mission.md states the project goal, scope, success criteria, and non-goals.

Goal
Rebuild a public technical profile through visible, agentic engineering work. Convert scattered real capability into recruiter-readable, peer-inspectable proof.

Scope
Public GitHub repository as source of truth; curated website as mirror; LinkedIn/social as campaign distribution; formal engineering documentation where warranted.

Success criteria
Portfolio of visible, documented artifacts; coherent technical narrative; demonstrated agentic workflow design; recruiter-facing landing page with AI-readable metadata; at least one formal case study.

Non-goals
Building a startup or product; becoming an AI influencer; mass-producing generic content; replacing human judgment.

4.2 brand.md

agents/rules/core/brand.md defines positioning and tone constraints.

The positioning: *systems-minded builder, technical generalist with unusual documentation and tooling depth, agentic workflow designer, engineer with range, judgment, and visible iteration discipline.*

Tone rules: technical, disciplined, direct; slightly artistic and young in energy; controlled sarcastic or provocative edge where appropriate; never chaotic, never self-sabotaging; never generic AI hype; never "just another junior" framing.

The language policy maps contexts to languages: English for core docs and code; English default for public content with Spanish when locally useful; Catalan permitted in formal LaTeX reports; Catalan as cultural accent, not default.

Forbidden framings: fake startup-founder noise, AI-influencer tone, empty self-description without evidence, CRUD-only backend identity box.

4.3 marketing.md

agents/rules/core/marketing.md establishes the campaign strategy.

Audience: recruiters first (fast proof of capability), engineering peers second (visible craft and systems thinking). Channels in priority order: repository (canonical source), website (curated

mirror), LinkedIn/social (selective distribution). Cadence rule: every sprint closure produces at least one narrative candidate suitable for social distribution.

Anti-patterns: generic self-promotion, AI-hype bandwagon posts, founder-LARP tone, content without linked evidence, duplicating identical text across channels.

4.4 constraints.md

agents/rules/core/constraints.md lists hard and soft constraints.

Hard: free/student-accessible tools only; model-agnostic (no vendor-specific prompt tricks in canonical files); human-reviewed truth (agents draft, humans approve); public inspectability (nothing operational hidden in private tooling); token-aware brevity (target ≤ 80 lines per file).

Soft: prefer checklists over prose; prefer small reusable templates; avoid repeated context across files; avoid decorative complexity.

4.5 truth-hierarchy.md

agents/rules/core/truth-hierarchy.md is the canonical source for the six-level priority table discussed in chapter 3. AGENTS.md cites it as a one-liner; this file contains the full resolution rules.

4.6 tool-policy.md

agents/rules/core/tool-policy.md lists the approved toolchain: GitHub (repository, issues, projects), GitHub Pages (hosting), GitHub Actions (CI/CD), Jekyll (static site generator), markdownlint-cli2 (Markdown linting), lychee (link checking), LaTeX via TeX Live (formal documents), and any capable LLM (model-agnostic execution).

Adding a new tool requires a pull request with justification, confirmation of a free or student-accessible tier, proof of no vendor lock-in, and an entry in the policy table.

Common pitfall: Do not introduce a new tool in a sprint and document it after the fact. The PR-first policy exists so that tool choices are auditable and reversible.

Chapter 5

Workflow Contracts: agents/rules/workflows/

Key takeaway: Five workflows define every type of work the system can perform. Each follows the same skeleton: Trigger, Inputs, Steps, Outputs, Exit Criteria. Agents read only the workflow they need.

Every workflow file under `agents/rules/workflows/` uses a shared five-section skeleton. This uniformity means an agent can parse any workflow without learning a new format.

5.1 plan.md — Design a new sprint

Trigger: User requests a new sprint plan, or the orchestrator identifies the need.

Steps: Select the next sprint seed from `agents/state/roadmap.md`; decompose into tasks with owning role, acceptance criteria, and expected output; define pair-check and backlog requirements; fill the sprint-output template; copy to `agents/state/active-sprint.md`; update `agents/state/current.md`.

Exit criteria: Every task has an owning role, acceptance criteria, and expected output; pair-check assignments exist for every non-trivial task; the plan is achievable within one sprint cycle.

5.2 sprint.md — Execute a planned sprint

Trigger: A populated `agents/state/active-sprint.md` with `status: planned`.

Steps: Orchestrator decomposes work → Developer agents execute → two fresh PairCheck agents review each non-trivial output → CI/CD integrates → Documentation emits docs → CommunityManager emits narrative → Orchestrator verifies closure.

Six closure artifacts (all required or explicitly waived):

1. Repository artifact(s) — committed code, configs, or docs.
2. Website / repo update trace.
3. Public-narrative decision.
4. Formal engineering documentation.
5. Condensed technical backlog.
6. Condensed narrative backlog.

5.3 hotfix.md — Small focused fix

Trigger: A small, focused change needed outside a full sprint cycle.

One Developer agent, narrow scope, one commit, minimal backlog note. No pair-check ceremony. **Escalation:** if scope expands during execution, convert to a sprint via the Plan workflow.

5.4 chat.md — Discussion only

Trigger: User initiates a discussion.

Read agents/rules/core/ and agents/state/current.md. No file edits except an optional summary written to `agents/state/summaries/`. Chat never silently promotes itself to a sprint.

5.5 system-review.md — Audit the system

Trigger: Scheduled review, user request, or detected inconsistency.

Audit docs/ for contradictions, dead routes, and token waste. Check that every path in AGENTS.md resolves. Review backlog and logs for recurring failures. Output: issue report with proposed diffs and migration notes if structure changes.

Common pitfall: Using the wrong workflow is a protocol violation. A "quick fix" that touches multiple concerns must be escalated from hotfix to sprint.

Chapter 6

Agent Roles: agents/rules/roles/

Key takeaway: Six roles define the division of labor. Each has a contract specifying what it reads, what it writes, what it must not do, and where it hands off. No role is self-approving.

6.1 Role contracts

Role	Reads	Writes	Must not
Orchestrator	Workflows, state, core	Sprint state, task contracts	Implement non-trivial code; skip pair-check
Developer	Task contract, core, repo	Code, configs, docs	Self-approve; work multiple tasks; modify core
PairCheck	Output, contract, core	Paircheck verdict	Modify output; communicate with other PairCheck
CI/CD	Approved outputs, workflows, tool-policy	Pipelines, status JSON, tags	Approve work; introduce unlisted tools
Documentation	Sprint outputs, mission, brand, templates	Docs (Markdown + LaTeX)	Publish directly; modify core; add decorative prose
CommunityManager	Sprint outputs, brand, marketing, templates	Social drafts	Publish without user approval; contradict brand

6.2 Interaction graph

Figure 6.1 shows the handoff paths between roles during a sprint workflow.

6.2.1 Key constraints across roles

- No role is self-approving: Developer outputs must pass through two independent PairCheck instances.
- PairCheck agents are always **fresh** — they cannot access prior review history or communicate with each other.
- Orchestrator delegates but never implements non-trivial artifacts directly.
- CommunityManager never publishes without explicit user approval.

Common pitfall: Allowing a role to both produce and approve its own output defeats the pair-check mechanism. If you find yourself skipping PairCheck "because it is simple," the hotfix workflow already exists for truly trivial changes.

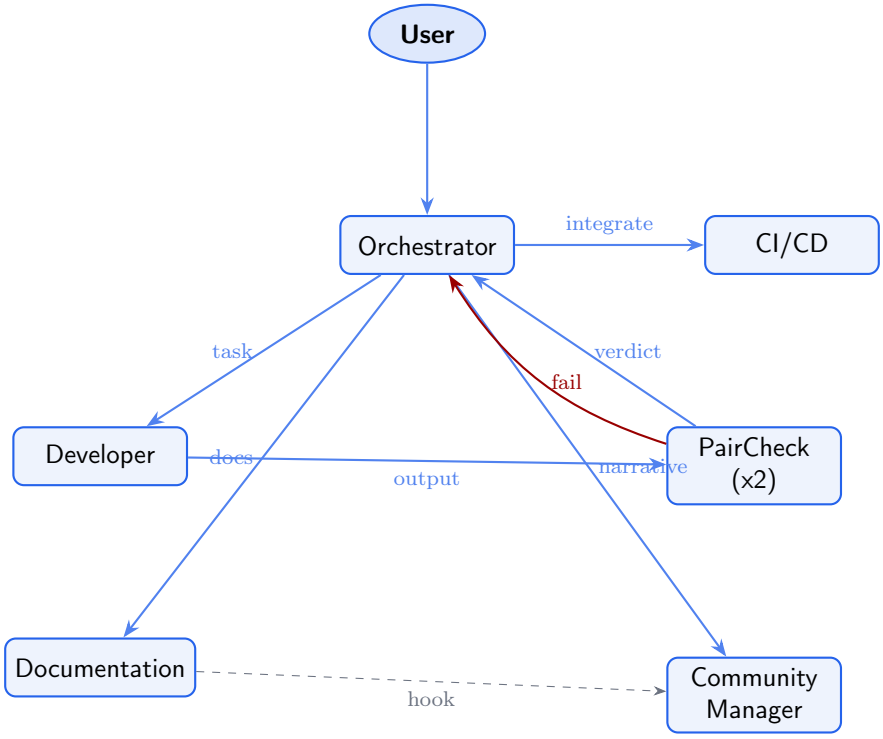


Figure 6.1: Agent interaction graph during a sprint workflow.

Chapter 7

Output Templates: agents/rules/templates/

Key takeaway: Four templates standardize every sprint output. Each is a short fillable Markdown contract. Agents fill templates rather than inventing formats, ensuring consistent traceability across sprints.

7.1 sprint-output.md

agents/rules/templates/sprint-output.md structures a sprint plan and closure record. Fields: Sprint ID, goal, status, task table (role, acceptance criteria, evidence link), pair-check assignment table, closure artifacts checklist, backlog deltas (technical + narrative), and CI trace.

The filled template is copied to agents/state/active-sprint.md when a sprint is planned, and updated in place as tasks complete.

7.2 paircheck-output.md

agents/rules/templates/paircheck-output.md records an independent review verdict. Fields: contract reference, verdict (pass/fail/partial), defects list, missing evidence, token-efficiency notes, mission alignment assessment, and an agent signature.

Two filled instances are saved to `agents/state/logs/` per reviewed output.

7.3 documentation-output.md

agents/rules/templates/documentation-output.md accompanies every documentation deliverable. Fields: artifact reference, date, rationale (why the work was done), mechanism (what was built), diagrams link, formal document link, and a public-narrative hook sentence suitable for the CommunityManager.

7.4 community-output.md

agents/rules/templates/community-output.md structures every public-facing artifact. Fields: source artifact, channel (LinkedIn/GitHub/Site), scheduled date, headline, body, and a checks section verifying tone consistency with agents/rules/core/brand.md, evidence linking, cross-channel originality, and technical identity support.

Common pitfall: Skipping the template and writing a free-form output breaks traceability. Every sprint artifact must be traceable back to a template instance saved in `agents/state/logs/`.

Chapter 8

State Machinery: agents/state/

Key takeaway: Four Markdown files and two log directories form the volatile memory of the system. They are updated every sprint or hotfix and are always subordinate to stable truth in `agents/rules/core/`.

8.1 Lifecycle overview

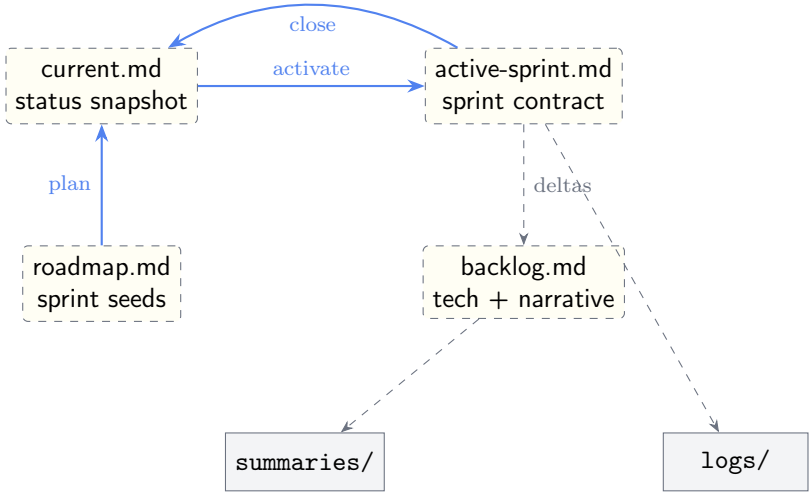


Figure 8.1: State file lifecycle during a sprint.

8.2 current.md

`agents/state/current.md` is a one-screen snapshot: active workflow, active sprint ID, blockers, and a table of the last three closures. It is the first volatile file any workflow reads.

8.3 active-sprint.md

`agents/state/active-sprint.md` holds the current sprint contract (filled from `agents/rules/templates/sprin` When no sprint is active, it reads `status: idle`. Task status is updated in place as work progresses.

8.4 roadmap.md

`agents/state/roadmap.md` lists 3–5 upcoming sprint seeds. Each seed becomes a full plan via the Plan workflow. Seeds are consumed (moved to active) and replenished after system reviews.

8.5 backlog.md

agents/state/backlog.md has two sections: *Technical backlog* (tickets with ID, description, origin, priority) and *Narrative backlog* (scrum-like notes tracking communication and strategy decisions).

8.6 logs/ and summaries/

`agents/state/logs/` accumulates pair-check outputs, CI traces, and per-sprint evidence files. `agents/state/summaries/` stores chat session summaries and periodic digests. Both are append-only during a sprint.

Common pitfall: Never treat data in `agents/state/` as canonical. If a volatile file says the mission is X but agents/rules/core/mission.md says Y, the core file wins unconditionally.

Chapter 9

Public Surfaces: content/, site/, data/

Key takeaway: Three content directories, a Jekyll scaffold, and two JSON files bridge the internal repository to its public-facing outputs.

9.1 content/ — artifact sources

`content/social/`
: Platform-adapted post drafts. Naming convention: `YYYY-MM-DD-<channel>-<slug>.md`.

`site/`
: Jekyll source for the public site.

`agents/reports/`
: Formal LaTeX engineering documents. This directory hosts the `tex/` subtree with shared preamble and per-sprint documents.

9.2 site/ — Jekyll scaffold

`site/` contains the canonical Jekyll site: `_config.yml` (base URL `/AgenticCareerBoost`, plugins: `jekyll-feed`, `jekyll-seo-tag`), `Gemfile`, `index.md` (landing page derived from the README), `_layouts/default.html` (semantic HTML, dark-mode friendly, no JS dependencies), and `projects/index.md` (reserved for per-repo subpages in a later sprint).

9.3 data/ — machine-readable state

`site/data/status.json` exports sprint status for the public site: sprint ID, workflow, last closure timestamp, artifacts array, and blockers. It is auto-generated by a GitHub Actions workflow whenever `agents/state/` changes.

`data/links.json` curates outbound links: GitHub profile, LinkedIn, legacy site, and this repository.

Figure 9.1: Jekyll site landing page (placeholder — replace with actual screenshot after first deploy).

Common pitfall: Do not edit `site/data/status.json` by hand. It is machine-generated by `export-status.yml`. Manual edits will be overwritten on the next push to `agents/state/`.

Chapter 10

CI/CD Automation: .github/

Key takeaway: Three GitHub Actions workflows, five issue templates, and a PR template enforce quality gates and automate the publish pipeline — all within GitHub's free tier.

10.1 Workflows

10.1.1 docs-lint.yml

Triggers on PR and push to main. Runs `markdownlint-cli2` over all `*.md` files using a `.markdownlint.jsonc` configuration that relaxes line-length rules for templates. Then runs `lychee` for link checking with retries and mail exclusion.

10.1.2 site-build.yml

Triggers on push to main affecting `site/`, or `README.md`. Uses the official GitHub Pages actions (`actions/configure-pages`, `actions/jekyll-build-pages`, `actions/deploy-pages`) with source at `./site/`. Permissions: `pages: write`, `id-token: write`.

10.1.3 export-status.yml

Triggers on push to main affecting `agents/state/` and on manual dispatch. A Python script (`.github/scripts/export_status.py`, ≤ 40 lines) parses front-matter from `agents/state/current.md` and `agents/state/active-sprint.md`, writes `site/data/status.json`, and auto-commits only if content changed.

10.1.4 latex-build.yml

Triggers on PR and push touching `agents/reports/tex/`. Uses `xu-cheng/latex-action` (TeX Live full) to run `latexmk -pdf` on every file under `sprints/`. Uploads compiled PDFs as workflow artifacts. Does not commit back or deploy to Pages — publication remains user-gated.

10.2 Issue templates

Five YAML-based issue forms standardize work requests: `sprint-task.yml`, `hotfix-task.yml`, `system-review.yml`, `website-update.yml`, and `social-post.yml`. Each maps directly to a workflow or agent role.

10.3 PR template

`.github/PULL_REQUEST_TEMPLATE.md` enforces a checklist: links to sprint or hotfix, pair-check references, closure artifacts updated, `agents/state/` updated, no orphan work.

Common pitfall: Adding a workflow that introduces a tool not listed in `agents/rules/core/tool-policy.md` is a policy violation. The tool must be approved via PR first, then the workflow can reference it.

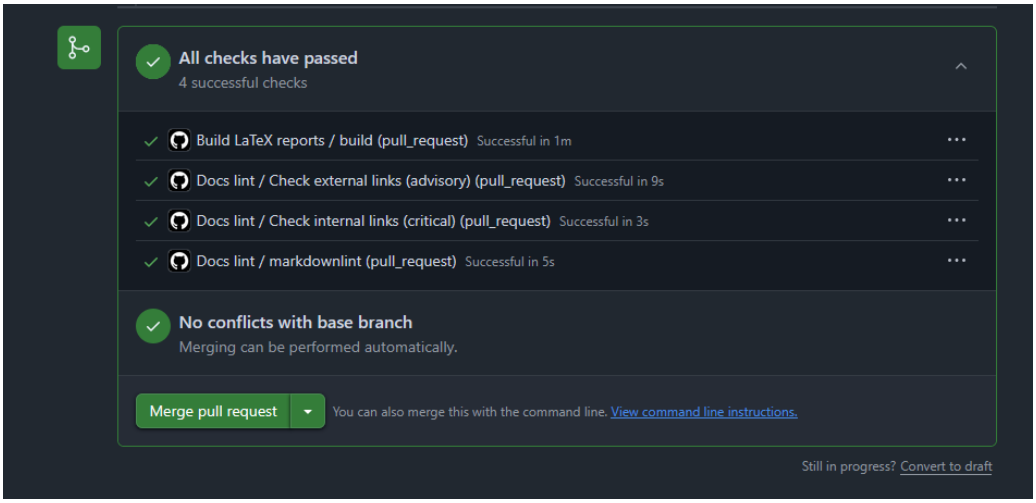


Figure 10.1: GitHub Actions workflow run showing all four pipelines (placeholder — replace after first CI run).

Chapter 11

Lessons and Invariants

Key takeaway: The bootstrap sprint established invariants that future sprints inherit. Breaking these invariants silently is the primary failure mode of agentic systems.

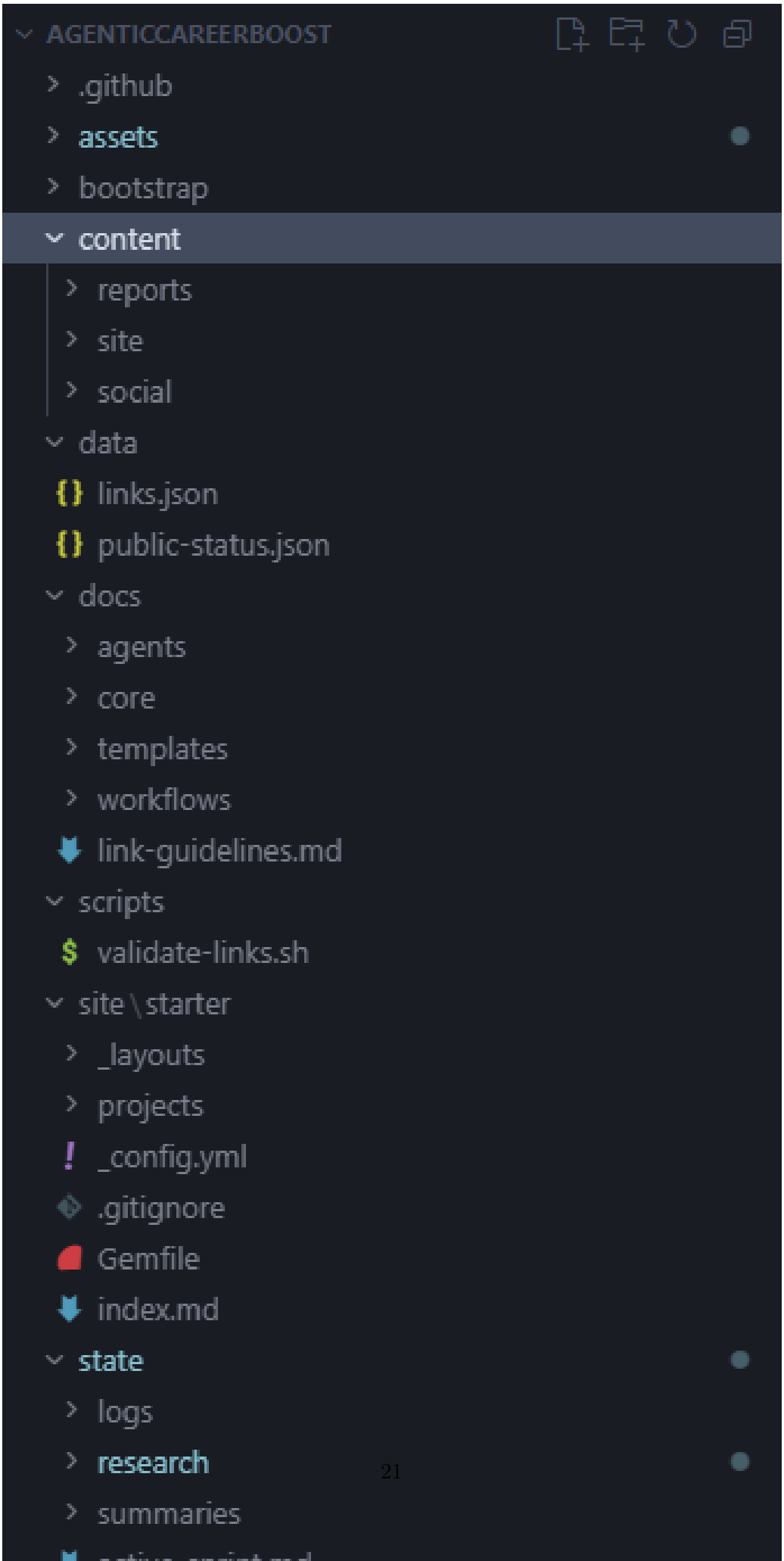
11.1 Design invariants

1. **Every file has exactly one owner.** Core files are owned by the user via system-review. State files are owned by the orchestrator. Templates are shared but version-controlled.
2. **No self-approval.** Every non-trivial output passes through two independent PairCheck instances before integration.
3. **No fabricated paths.** If an agent references a file that does not exist, it must stop and ask rather than create a plausible-sounding alternative.
4. **No mega-prompts.** The system never concatenates all files into a single context. Agents load what they need, guided by `AGENTS.md`.
5. **Public inspectability.** The process is part of the proof. Nothing operational is hidden in private tooling.
6. **Token-aware brevity.** Files target ≤ 80 lines. Many are under 30. Longer files require justification.

11.2 Forbidden patterns

- Storing state in code comments or commit messages instead of `agents/state/`.
- Duplicating truth across files instead of linking.
- Skipping PairCheck because "the change is small" — use the hotfix workflow instead.
- Adding tools, then documenting them after the fact.
- Using model-specific prompt syntax in any file under `docs/`.

Common pitfall: The most dangerous failure mode is a "temporary" exception to an invariant that becomes permanent. If an invariant needs to change, use the system-review workflow — never silently work around it.


AGENTICCAREERBOOST
.github
assets
bootstrap
content
reports
site
social
data
links.json
public-status.json
docs
agents
core
templates
workflows
link-guidelines.md
scripts
validate-links.sh
site \ starter
_layouts
projects
_config.yml
.gitignore
Gemfile
index.md
state
logs
research
summaries

Appendix A

File Manifest

Key takeaway: Every file in the repository after the bootstrap sprint, with its purpose and approximate size.

Path	Purpose	Lines
AGENTS.md	Agent entrypoint / dispatch	~65
README.md	Public human entrypoint	~70
agents/rules/core/mission.md	Goal, scope, success criteria	~29
agents/rules/core/brand.md	Positioning, tone, language policy	~36
agents/rules/core/marketing.md	Campaign strategy	~33
agents/rules/core/constraints.md	Hard and soft constraints	~23
agents/rules/core/truth-hierarchy.md	Conflict resolution order	~20
agents/rules/core/tool-policy.md	Approved toolchain	~24
agents/rules/workflows/plan.md	Sprint planning workflow	~37
agents/rules/workflows/sprint.md	Sprint execution workflow	~42
agents/rules/workflows/hotfix.md	Focused fix workflow	~35
agents/rules/workflows/chat.md	Discussion workflow	~31
agents/rules/workflows/system-review.md	System audit workflow	~34
agents/rules/roles/orchestrator.md	Orchestrator role contract	~36
agents/rules/roles/developer.md	Developer role contract	~32
agents/rules/roles/paircheck.md	PairCheck role contract	~38
agents/rules/roles/cicd.md	CI/CD role contract	~34
agents/rules/roles/documentation.md	Documentation role contract	~33
agents/rules/roles/community-manager.md	CommunityManager role contract	~32
agents/rules/templates/sprint-output.md	Sprint plan/closure template	~48
agents/rules/templates/paircheck-output.md	Review verdict template	~25
agents/rules/templates/documentation-output.md	Documentation template	~29
agents/rules/templates/community-output.md	Social artifact template	~26
agents/state/current.md	Active workflow snapshot	~12
agents/state/active-sprint.md	Current sprint contract	~7
agents/state/roadmap.md	Upcoming sprint seeds	~11
agents/state/backlog.md	Technical + narrative backlog	~18

Additional infrastructure files (`.github/`, `site/`, `data/`, `bootstrap/`, `assets/`) are documented in their respective chapters above.